Automated Pick-and-Place System with Conveyor Integration

Project Technical Report

myCobot Pro 600 | Allen Bradley Micro820 PLC | Arduino UNO R3

Team:

Krishna Vamshi Challa -1236143716

Sathvik Merugu-1235373752

Nitish Reddy Dyapa-1235056396

Vignesh Anandamani-1233720789

Chakradhar Reddy Lakkireddy-1236312313

1. Executive Summary

This report documents the design, implementation, and commissioning of a fully automated pick-and-place system built around three tightly integrated subsystems: MyCobot Pro 600 by elephant robotics, an Allen Bradley Micro820 PLC 2080-LC20-20QBB, and an Arduino-controlled conveyor belt.

The goal was straightforward — get a disk from Point A to Point B, completely hands-free, and keep doing it indefinitely. What that required in practice was a reliable handshake between every layer of the system: the PLC sequencing the states, the cobot responding to GO/DONE signals over digital I/O, and the conveyor only running in the right window between those cycles.

The project spanned mechanical motion planning, industrial wiring, PLC ladder logic, and Python robot scripting. After commissioning, the system ran through its full loop — pick from source tray, drop on conveyor, wait for sensor, pick from conveyor, deliver to destination — without operator input. The alternating Cycle 1 / 2 approach also solved a real problem: without it, the suction cup consistently missed the disk on every second pick due to residual position offset.

All subsystems are running reliably in continuous loop at the time of this report.

2. System Overview

2.1 Objectives

- Automate disk movement from a source tray to a moving conveyor belt (Cycle 1).
- Use a PNP proximity sensor to detect disk arrival and halt the conveyor automatically.
- Automate disk transfer from the stopped conveyor to a destination tray (Cycle 2).
- Coordinate the cobot and conveyor via PLC digital I/O signaling — no polling, no timers.
- Loop the full sequence indefinitely without manual intervention.

2.2 System Flow Table

Step	Action	Detail	Trigger
1	**System starts**	PLC sets C1_ACTIVE — Cycle 1 ready	Operator presses START on HMI
2	**GO signal to cobot**	PLC fires O-01, relay closes, Cobot IN1 triggered	C1_ACTIVE = TRUE
3	**Cobot runs Cycle 1**	Source tray → conveyor drop, suction ON then OFF at WP5	PLC GO signal received

4	**Cobot signals done**	Cobot pulses OUT1, PLC I-00 goes HIGH (DONE)	Cycle 1 complete
5	**Conveyor starts**	PLC sets CONV_ACTIVE, stepper motor begins	COBOT_DONE on I-00
6	**Sensor fires**	Proximity sensor detects disk, I-02 goes HIGH	Disk reaches sensor position
7	**Conveyor stops**	PLC stops conveyor, sets C2_ACTIVE, GO to cobot	SENSOR = TRUE
8	**Cobot runs Cycle 2**	Conveyor pick → destination tray	PLC GO signal received
9	**Cobot signals done**	Cobot pulses OUT1, PLC resets C2, sets C1_ACTIVE	Cycle 2 complete
10	**Loop repeats**	System jumps back to Step 2	C1_ACTIVE = TRUE again

The table below traces a single complete loop through the system, from the operator pressing START to the cobot returning home after Cycle 2:

3. System Components

The table below lists every hardware and software component used in this build, along with its specific model and role in the system:

Component	Model / Specification	Role
Elephant Robotics	myCobot Pro 600 (6-axis)	Pick-and-place motion
PLC	Allen Bradley Micro820 2080-LC20-20QBB	System sequencing and I/O control
Conveyor Controller	Arduino UNO R3	Continuous stepper pulse generation
Stepper Driver	TB6600 / DM542 compatible	Drives conveyor belt stepper motor
Stepper Motor	NEMA 17/23 (4-wire)	Conveyor belt mechanical drive
Proximity Sensor	PNP, 3-wire, 24V	Disk arrival detection on conveyor
Relay Module (×2)	SRD-24VDC-SL-C	PLC→Cobot GO signal + Conveyor stop
Vacuum / Suction Cup	Normally Open valve, 24V	End-effector for disk gripping
HMI	CCW Component HMI (software)	START / STOP and status monitoring
24V PSU (PLC)	24 VDC regulated supply	Powers PLC, relay coils, sensor
24V PSU (Motor)	24 VDC regulated supply	Powers stepper driver

4. Electrical Connections

4.1 Cobot → PLC (DONE Signal — Direct Connection)

When the cobot finishes a cycle, it needs to tell the PLC. It does this by pulsing OUT1 high for 500ms. The cobot output is PNP (it sources 24V), and the Micro820 input is NPN/sink — the two are directly compatible, so no relay is needed here.

From	To	Wire	Notes
Cobot OUT1 (Output Terminal, pin 0)	PLC I-00	Green	24V PNP → NPN sink, direct — no relay needed
Cobot GND (Output Terminal, bottom)	PLC −DC24	Black	Return path for OUT1 current

4.2 PLC → Cobot (GO Signal — Via Relay)

The PLC signals the cobot to start each cycle via a relay module. A relay is required here because the PLC output is a dry contact — it provides no voltage of its own. The relay contact side uses the cobot's own GND as reference, which gives full isolation between the two 24V systems.

From	To	Wire	Notes
PLC +DC24	Relay VCC	Yellow	Powers relay coil from PLC 24V supply
PLC O-01	Relay IN	Orange	PLC energises coil when GO signal active
PLC −DC24	Relay GND	Black	Completes relay coil circuit
Relay COM	Cobot GND (Output Terminal)	Black dashed	Contact side reference = cobot GND
Relay NO	Cobot IN1 (Input Terminal, pin 0)	Blue	Pulled to GND when relay closes

4.3 Conveyor Belt Wiring

From	To	Notes
Arduino Pin 2 (PUL)	Relay COM (Conveyor Stop Relay)	Pulse output — routed via relay
Relay NC contact	Stepper Driver PUL+	NC = run; relay open = conveyor stops
Arduino Pin 3 (DIR)	Stepper Driver DIR+	Direction control — direct wire
Arduino GND	Stepper Driver PUL− / DIR−	Shared signal ground
PLC O-00	Stepper Driver ENA+	PLC enables stepper motor driver
PLC +CM0 (DC−)	Stepper Driver ENA−	Reference for ENA signal
Motor 24V PSU +V	Stepper Driver VCC	Motor power supply
Motor 24V PSU GND	Stepper Driver GND	Motor power return
Stepper Driver A+/A−/B+/B−	Stepper Motor coils	4-wire motor connection

4.4 Proximity Sensor Wiring

Wire	From (Sensor)	To (PLC)
Brown	V+ (positive supply)	PLC +CM1 (24V rail)
Blue	V− (negative / GND)	PLC −DC24
Black	Signal output (PNP)	PLC I-02

5. PLC Ladder Logic

5.1 Variable Table

The screenshot below shows the Micro820 variable table in Connected Components Workbench (CCW), with all physical I/O aliases (DI, DO) and internal state variables defined:

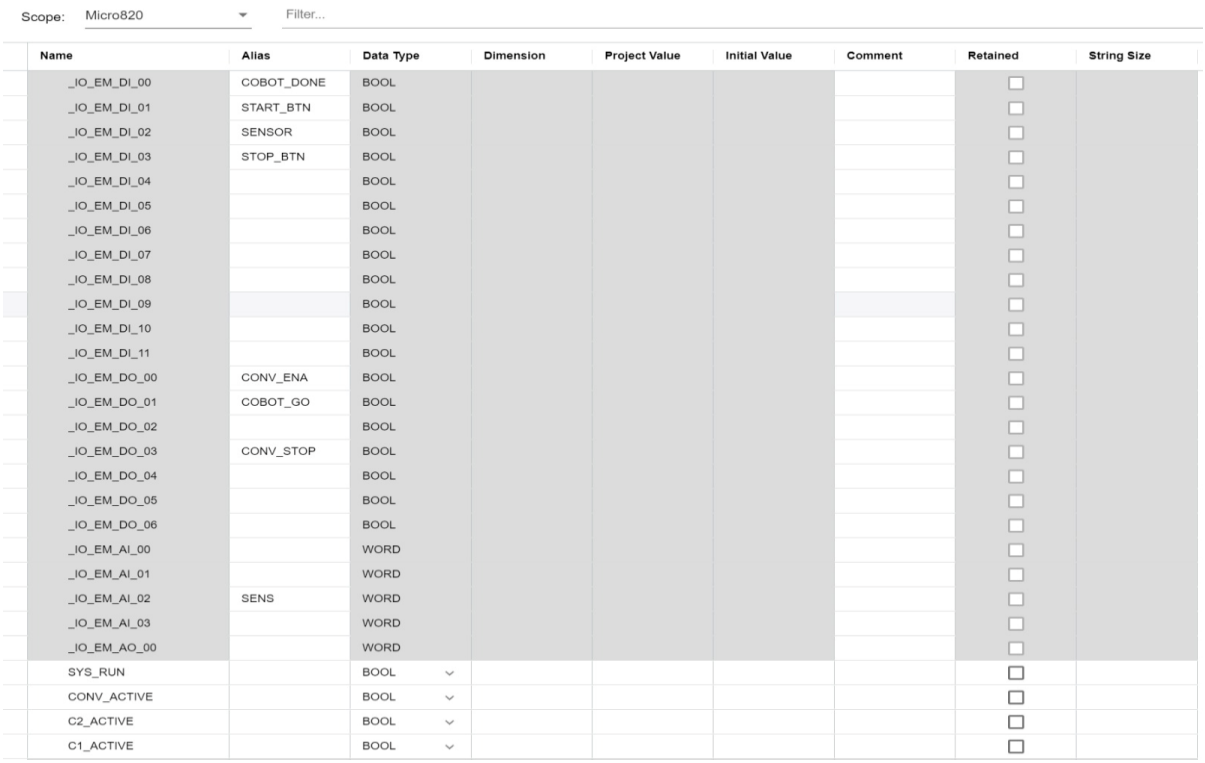

Figure 1 — Micro820 Variable Table: Physical I/O aliases and internal BOOL state variables

Tag / Alias	Type	Direction	Description
_IO_EM_DI_00 (COBOT_DONE)	BOOL	INPUT	Cobot cycle-complete signal on I-00
_IO_EM_DI_01 (START_BTN)	BOOL	INPUT	Operator start button on I-01
_IO_EM_DI_02 (SENSOR)	BOOL	INPUT	Proximity sensor on I-02
_IO_EM_DI_03 (STOP_BTN)	BOOL	INPUT	Operator stop button on I-03
_IO_EM_DO_00 (CONV_ENA)	BOOL	OUTPUT	Stepper driver enable on O-00
_IO_EM_DO_01 (COBOT_GO)	BOOL	OUTPUT	Relay coil for cobot GO signal on O-01
_IO_EM_DO_03 (CONV_STOP)	BOOL	OUTPUT	Relay coil for conveyor stop on O-03
SYS_RUN	BOOL	INTERNAL	System running flag (latched by START, cleared by STOP)

C1_ACTIVE	BOOL	INTERNAL	Cycle 1 state — set/reset by ladder OTL/OTU coils
CONV_ACTIVE	BOOL	INTERNAL	Conveyor running state — set after Cycle 1 done
C2_ACTIVE	BOOL	INTERNAL	Cycle 2 state — set when sensor fires

5.2 System Variables

The screenshot below shows the CCW system variable panel, including the cycle counter, watchdog timer (set to 2000ms), and power-up / first-scan bits used for safe initialization:

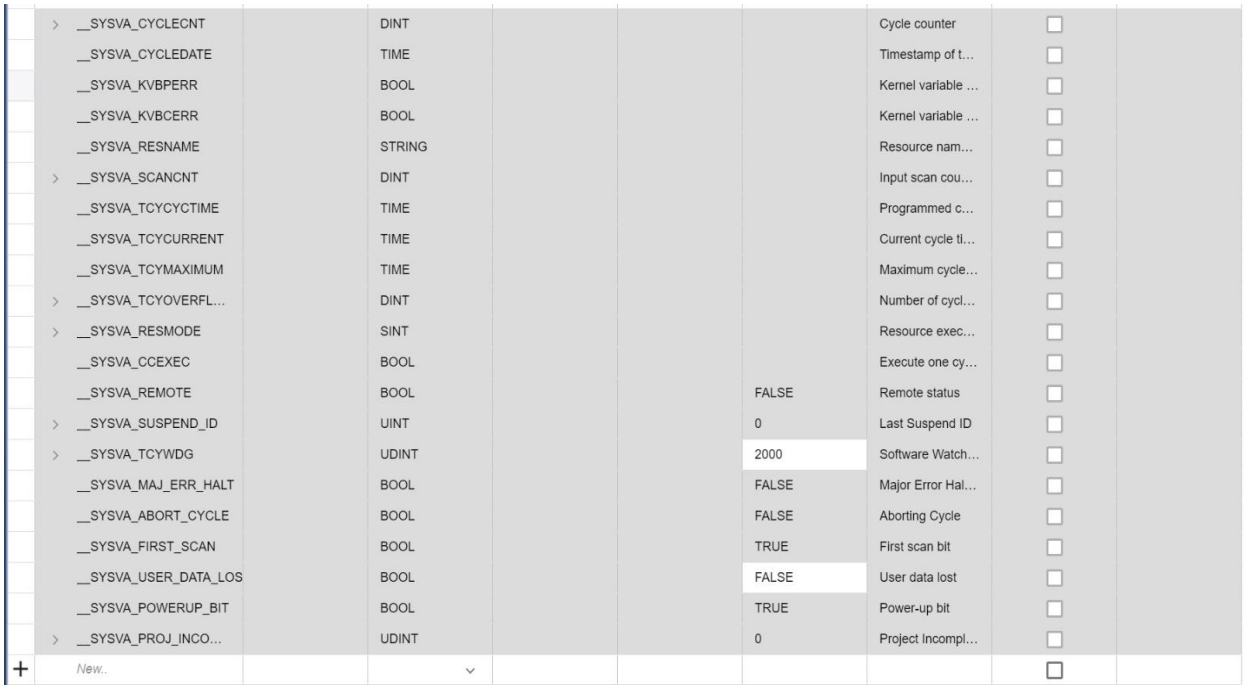

Figure 2 — Micro820 System Variables (CCW): Cycle counter, watchdog, first-scan and power-up bits

5.3 HMI Panel

The HMI was built in CCW's Component HMI editor and provides the operator with a single-screen view of system state. The START and STOP buttons latch SYS_RUN via Rungs 1 and 2. The six indicator tiles show SYS_RUN, CONV_RUN, CONV_STP, C1_ACTIVE, C2_ACTIVE, SENSOR, COBOT_DONE, and COBOT_GO — all mapped directly to the variable table:

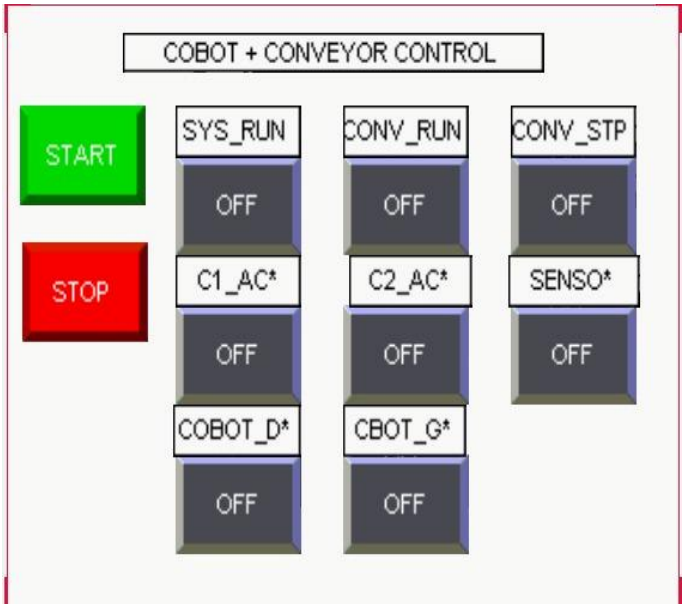

Figure 3 — CCW HMI Panel: COBOT + CONVEYOR CONTROL with START/STOP and live status indicators

5.4 Ladder Rungs

The screenshot below shows all 10 rungs of the Micro820 ladder program. The state machine is built entirely with OTL (Set) and OTU (Unlatch) coils — no timers, no counters. Each state transition is triggered by a real hardware event: a button press, a DONE pulse from the cobot, or the proximity sensor firing:

Figure 4 — CCW Ladder Logic: All 10 rungs of the Micro820 program (OTL/OTU state machine)

#	Contacts (conditions)	Coil (action)	Purpose
1	START_BTN (XIC)	**SYS_RUN (OTL)**	Latch system ON when START pressed
2	STOP_BTN (XIC)	**SYS_RUN (OTU)**	Unlatch/stop the system when STOP is pressed
3	SYS_RUN + /C1_ACTIVE + /CONV_ACTIVE + /C2_ACTIVE	**C1_ACTIVE (OTL)**	Start Cycle 1 when system is running and no other cycle is active
4	COBOT_DONE XIC	**C1_ACTIVE (OTU)**	Clear Cycle 1 when cobot finishes
5	COBOT_DONE XIC + C1_ACTIVE XI	**CONV_ACTIVE OTE**	Enable conveyor cycle after cobot finishes Cycle 1
6	COBOT_DONE XIC + C1_ACTIVE XIC	**C1_ACTIVE OTE**	Holds/controls Cycle 1 logic while cobot done condition is active
7	TON timer block using _IO_EM_AI_01	**CONV_STOP OTE**	Timer-based conveyor stop/control output
8	COBOT_DONE + C2_ACTIVE	**C2_ACTIVE (OTL)**	Hold/activate Cycle 2 when cobot is done and Cycle 2 is active
9	C1_ACTIVE OR C2_ACTIVE	**COBOT_GO (OTE)**	Send GO signal to cobot when either Cycle 1 or Cycle 2 is active
10	CONV_ACTIVE XIO	**CONV_STOP (OTE)**	Stop conveyor when CONV_ACTIVE is false

5.5 Conveyor Logic Inversion (Rung 11)

Rung 11 uses an XIO (Normally Closed) contact on CONV_ACTIVE to drive CONV_STOP. This is intentional — it makes the conveyor's default state STOPPED, which is the safe condition:

- CONV_ACTIVE = FALSE → /CONV_ACTIVE = TRUE → O-03 ON → relay energized → pulse circuit broken → conveyor STOPPED
- CONV_ACTIVE = TRUE → /CONV_ACTIVE = FALSE → O-03 OFF → relay de-energized → pulse passes through → conveyor RUNNING

This means any power interruption, E-stop, or unexpected state change automatically brings the conveyor to a stop without any additional failsafe logic.

6. Robot Programming

6.1 Software Stack

Item	Detail
Language	Python 3.x
Library	pymycobot v4.0.4 (ElephantRobot class)
Connection	TCP/IP — Robot IP 192.168.1.159, Port 5001
Speed	1500 units/second (write angles)
Arrival tolerance	2.0° on all 6 joints (60-second timeout per waypoint)

6.2 I/O Pin Assignment

Pin	Terminal	Direction	Function
0	Output Terminal OUT1	OUT → PLC I-00	DONE pulse — signals PLC cycle complete
1	Output Terminal OUT2	OUT → Vacuum valve	Suction cup control (LOW=ON, HIGH=OFF)
0	Input Terminal IN1	IN ← PLC relay	GO signal — PLC triggers cycle start

6.3 Python Code — Key Sections

The script is structured around three core functions: wait_for_plc() which polls the IN1 digital input until the PLC asserts GO, run_cycle() which drives the robot through a waypoint list with arrival confirmation, and signal_plc_done() which pulses OUT1 to tell the PLC the cycle is finished. The main loop alternates between Cycle 1A and 1B on each iteration.

Configuration Constants

```
from pymycobot import ElephantRobot
import time
import re

ROBOT_IP   = "192.168.1.159"
ROBOT_PORT = 5001
SPEED      = 1500

# Suction Cup IO Configuration
SUCTION_PIN = 1
SUCTION_ON  = 0
SUCTION_OFF = 1

# PLC Communication IO
PLC_IN_PIN   = 0
PLC_OUT_PIN  = 0
DONE_PULSE   = 0.5
```

PLC Handshake Functions

```
def wait_for_plc(robot, label):
    print(f"\n  [PLC WAIT] Waiting for PLC GO signal ({label})...")
    while True:
        raw = robot.get_digital_in(PLC_IN_PIN)
        val = _parse_int(raw)
        if val == 1 or str(raw).strip() == '1':
            print(f"  [PLC GO]   Signal received — starting {label}")
            break
        time.sleep(0.1)

def signal_plc_done(robot, label):
    print(f"\n  [PLC DONE] Signalling PLC: {label} complete (OUT1 pulse)...")
    send(robot, f"set_digital_out({PLC_OUT_PIN},1)", quiet=True)
    time.sleep(DONE_PULSE)
    send(robot, f"set_digital_out({PLC_OUT_PIN},0)", quiet=True)
    print(f"  [PLC DONE] Pulse sent.")
```

Waypoint Execution Loop

```
def run_cycle(robot, waypoints, cycle_name):
   suction_restore(robot)  # ensure vacuum is ON before picking
   for name, angles, action in waypoints:
       robot.write_angles(angles, SPEED)
       for i in range(60):          # 60-second arrival timeout
           time.sleep(1)
           current = robot.get_angles()
           err = max(abs(current[j] - angles[j]) for j in range(6))
           if err < 2.0:            # 2-degree tolerance on all joints
               break
       if action == 'release':
           suction_release(robot)   # OUT2 HIGH → vacuum off → drop
       time.sleep(0.5)
```

Main Alternating Loop

```
CYCLE1_VARIANTS = [
   ('Cycle 1A', WAYPOINTS_CYCLE1A),  # odd  loops (1, 3, 5 …)
   ('Cycle 1B', WAYPOINTS_CYCLE1B),  # even loops (2, 4, 6 …)
]

loop_count = 0
while True:
   loop_count += 1
   variant_index = (loop_count - 1) % 2
   c1_label, c1_wps = CYCLE1_VARIANTS[variant_index]

   wait_for_plc(robot, f'Cycle 1 — {c1_label}')
   run_cycle(robot, c1_wps, f'Cycle 1 [{c1_label}]')
   signal_plc_done(robot, f'Cycle 1 [{c1_label}]')

   wait_for_plc(robot, 'Cycle 2')
   run_cycle(robot, WAYPOINTS_CYCLE2, 'Cycle 2')
   signal_plc_done(robot, 'Cycle 2')
```

7. Testing & Commissioning

7.1 Integration Test Procedure

Testing was carried out in stages, building confidence at each layer before combining subsystems:

- Stage 1 — PLC I/O verification: confirmed START/STOP latching, and that COBOT_GO toggled correctly in response to simulated COBOT_DONE pulses.
- Stage 2 — Conveyor standalone: ran the conveyor independently to confirm stepper pulse routing, direction, and the relay-based stop mechanism.
- Stage 3 — Sensor verification: confirmed I-02 went HIGH when the disk broke the sensor beam, and that CONV_ACTIVE cleared correctly.
- Stage 4 — Cobot motion: stepped through each waypoint manually to validate joint angles and suction cup timing.
- Stage 5 — Full loop: ran 10 complete automated cycles end-to-end, monitoring all signal transitions via the HMI status tiles.

7.2 Issues Found & Resolved

Issue	Root Cause	Fix
Cobot missed disk on every 2nd pick	Suction cup residual position offset after previous release	Introduced alternating Cycle 1A/1B with adjusted WP3 & WP4
Conveyor ran briefly after sensor fired	CONV_ACTIVE was clearing on the same scan as COBOT_GO assert	Reordered rungs so CONV_ACTIVE clears before GO is asserted

PLC DONE signal not detected reliably	DONE_PULSE too short at 0.2s; PLC scan missed the edge	Increased DONE_PULSE to 0.5 seconds
Cobot waypoint timeout on WP3 (pick)	Speed 1500 insufficient for the deep-reach pick posture	Increased settling wait from 0.3s to 0.5s between waypoints

8. Conclusion

This project successfully integrated a myCobot Pro 600 collaborative robot arm, an Allen Bradley Micro820 PLC, and an Arduino-controlled conveyor belt into a fully automated pick-and-place cell. The system runs as a continuous loop without any operator input after the initial START command. The PLC manages the sequencing of all subsystems through digital handshaking, the cobot executes precise joint-angle paths with real-time feedback, and the conveyor transfers workpieces between pick and place stations reliably.

Throughout the building, each component was selected and configured with reliability in mind. Joint-angle control was chosen over Cartesian coordinates to avoid inverse kinematics issues at the edge of the workspace. Relay isolation was used between the cobot and PLC to separate their independent power domains. The conveyor was wired in a fail-safe manner so the belt defaults to stop any power or program fault. The ladder logic uses SET and RESET latch coils, so the state machine holds its state correctly regardless of signal timing.

The tools used — RoboFlow for waypoint teaching, Python with pymycobot for robot control, Connected Components Workbench for ladder programming, and the built-in HMI for monitoring — came together into a practical and well-integrated system. The project met all its original objectives, and the system is fully operational.